

A Technical Introduction to Independent component analysis

Section 1

$$x_i = \bigvee_{j=1}^n (g_{ij} \wedge y_j), i = 1, 2, \dots, m, \quad \{ \displaystyle x_{ij} = \bigvee_{j=1}^n (g_{ij} \wedge y_j), i=1,2,\ldots,m, \}$$

Section 2

Defining component independence ICA finds the independent components (also called factors, latent variables or sources) by maximizing the statistical independence of the estimated components. We may choose one of many ways to define a proxy for independence, and this choice governs the form of the ICA algorithm. The two broadest definitions of independence for ICA are

Section 3

$$J(x) = \frac{1}{12} (E(x^3))^2 + \frac{1}{48} (kurt(x))^2 \quad \{ \displaystyle J(x) = \frac{1}{12} (E(x^3))^2 + \frac{1}{48} (kurt(x))^2 \}$$

Section 4

Binary ICA A special variant of ICA is binary ICA in which both signal sources and monitors are in binary form and observations from monitors are disjunctive mixtures of binary independent sources. The problem was shown to have applications in many domains including medical diagnosis, multi-cluster assignment, network tomography and internet resource management. Let $x_1, x_2, \dots, x_m$ be the set of binary variables from m monitors and $y_1, y_2, \dots, y_n$ be the set of binary variables from n sources. Source-monitor connections are represented by the (unknown) mixing matrix G , where $g_{ij} = 1$ indicates that signal from the i -th source can be observed by the j -th monitor. The system works as follows: at any time, if a source i is active ($y_i = 1$) and it is connected to the monitor j ($g_{ij} = 1$) then the monitor j will observe some activity ($x_j = 1$). Formally we have:

Section 5

$$p(y) = p(x) \mid \partial y \partial x = p(x) \mid W \quad \{ \displaystyle p(y) = p(x) \mid \partial y \partial x = p(x) \mid W \}$$

Section 6

where $\wedge$ is Boolean AND and $\vee$ is Boolean OR. Noise is not explicitly modelled, rather, can be treated as independent sources. The above problem can be heuristically solved by assuming variables are continuous and running FastICA on binary observation data to get the mixing matrix G (real values), then apply round number techniques on G to obtain the binary values. This approach has been shown to produce a highly inaccurate result. Another method is to use dynamic programming: recursively breaking the observation matrix X into its sub-matrices and run the inference algorithm on these sub-matrices. The key observation which leads to this algorithm is the sub-matrix X_0 of X where $x_{ij} = 0, \forall j$ corresponds to the unbiased observation matrix of hidden components that do not have connection to the i -th monitor. Experimental results from show that this approach is accurate under moderate noise levels. The Generalized Binary ICA framework introduces a broader problem formulation which does not necessitate any knowledge on the generative model. In other words, this method attempts to decompose a source into its independent components (as much as possible, and without losing any information) with no prior assumption on the way it was generated. Although this problem appears quite complex, it can be accurately solved with a branch and bound search tree algorithm or tightly upper bounded with a single multiplication of a matrix with a vector.

Section 7

weighted by the mixing weights $a_{i,k}$. The same generative model can be written in vector form as $x = \sum_{k=1}^n s_k a_k$, where the observed random vector x is represented by the basis vectors $a_k = (a_{1,k}, \dots, a_{m,k})^T$. The basis vectors a_k form the columns of the mixing matrix $A = (a_1, \dots, a_n)$ and the generative formula can be written as $x = A s$, where $s = (s_1, \dots, s_n)^T$. Given the model and realizations (samples) $x_1, \dots, x_N$ of the random vector x , the task is to estimate both the mixing matrix A and the sources s . This is done by adaptively calculating the w vectors and setting

up a cost function which either maximizes the non-gaussianity of the calculated $s_k = \mathbf{w}^T \mathbf{x}$ or minimizes the mutual information. In some cases, a priori knowledge of the probability distributions of the sources can be used in the cost function. The original sources $\mathbf{s}$ can be recovered by multiplying the observed signals $\mathbf{x}$ with the inverse of the mixing matrix $\mathbf{W} = \mathbf{A}^{-1}$, also known as the unmixing matrix. Here it is assumed that the mixing matrix is square ($n = m$). If the number of basis vectors is greater than the dimensionality of the observed vectors, $n > m$, the task is overcomplete but is still solvable with the pseudo inverse.

Section 8

where η is a small constant to guarantee that $\mathbf{w}$ converges to the optimal solution. After each update, we normalize $\mathbf{w}_{new} = \mathbf{w}_{new} / \|\mathbf{w}_{new}\|$, and set $\mathbf{w}_{old} = \mathbf{w}_{new}$, and repeat the updating process until convergence. We can also use another algorithm to update the weight vector $\mathbf{w}$. Another approach is using negentropy instead of kurtosis. Using negentropy is a more robust method than kurtosis, as kurtosis is very sensitive to outliers. The negentropy methods are based on an important property of Gaussian distribution: a Gaussian variable has the largest entropy among all continuous random variables of equal variance. This is also the reason why we want to find the most nongaussian variables. A simple proof can be found in Differential entropy.